autotest-lab ・ 一般論

GUI自動操作の技術

Web・iOS・Android・Mac・Windows を一望する

主要なライブラリとツールを、対象OS・書く言語・画面の掴み方・AIエージェントとの相性で比べる。

公開情報にもとづく一般論。各ツールの最新状況は公式ドキュメントで確認すること。

全体像

画面の「掴み方」は3通り

A. アプリの内側から

テストがアプリと同じプロセスで動き、部品を直接たどる。
例：Espresso、Flutter integration_test

強み 速い・待ちが安定
弱み アプリの外（通知・設定・他アプリ）は触れない

B. 部品の木（DOM・アクセシビリティ）

OSやブラウザが公開する部品の木から、IDや名前で探して操作する。
例：Selenium、Playwright、Appium、UI Automator、XCUITest、pywinauto、macOSのAX API

強み 主流。テキストで扱えるのでAIとも相性がよい
弱み IDの付け方で安定度が決まる

C. 画像と座標

スクリーンショットを見て、画像認識や座標でクリックする。
例：PyAutoGUI、SikuliX、AIのcomputer use

強み どのOS・どのアプリでも動く
弱み 遅い。解像度やテーマの違いで壊れやすい

「1つのスクリプトで複数OS」を狙う方法：Appium（OSごとのドライバを差し替え）、Maestro（YAMLのフロー）、Robot Framework（キーワードの層でライブラリを包む）、画像ベース。ただし部品の名前やIDはOSごとに違うため、共通化できるのは「手順」までで、「部品の指定」はOS別になりやすい。

主要ツール

比較表 1：Web とモバイル

ツール	対象	書く言語	仕組み	特徴・注意
Selenium	Web（主要ブラウザ）	Java, Python, C#, JS, Ruby	W3C標準のWebDriver。各ブラウザのドライバ経由。WebDriver BiDiへ移行中	最も歴史が長く情報が多い。待ち処理は自分で書く場面が多い
Playwright	Web（Chromium・Firefox・WebKit）	TS/JS, Python, Java, .NET	ブラウザを直接制御するライブラリ＋テストランナー（Microsoft）	自動待機、録画からのコード生成、トレース。ネイティブアプリは対象外
Cypress	Web（主にChromium系・Firefox）	JS/TS	ブラウザの中でテストを実行	開発者体験が良い。複数タブなどに制約
Espresso	Android（自アプリ内）	Kotlin, Java	アプリと同じプロセスで動く（掴み方A）	速く安定。アプリの外は操作できない
UI Automator	Android（端末全体）	Kotlin, Java	アクセシビリティ情報（掴み方B）	通知・設定・他アプリも操作できる
XCUITest	iOS（macOSも）	Swift, Objective-C	Appleの XCTest。アクセシビリティ情報	Xcodeが必要＝Macが必要
Appium	iOS・Android・Mac・Windows ほか	Java, Python, JS, C#, Ruby	WebDriverの口＋OSごとのドライバ（UiAutomator2、XCUITest、Mac2、Windows）	1つのAPIで複数OS。下で動くのは各OSの公式の仕組み
Maestro	iOS・Android・Web	YAML	宣言的なフローを書く	書くのが簡単。複雑な分岐や計算は苦手

主要ツール

比較表 2：デスクトップと、OSを横断するもの

ツール	対象	書く言語	仕組み	特徴・注意
pywinauto	Windows	Python	UI Automation（UIA）または Win32 API	手軽。UIAは遅い部品や取りにくい部品がある
FlaUI	Windows	C#（.NET）	UIA	.NETでの定番
WinAppDriver	Windows	WebDriver対応の各言語	UIAをWebDriverの口で公開（Microsoft）	本家の開発は停滞気味。Appium経由で使われることが多い
macOS AX API	macOS	Python（pyobjc）, Swift	AXUIElement（アクセシビリティ）	このリポジトリの macos/ の実験。権限の許可が必要
Appium Mac2	macOS	Appiumの各言語	XCTestを使うドライバ	Xcodeが必要
AppleScript / JXA	macOS	AppleScript, JavaScript	System Events 経由でアクセシビリティ	手軽だが遅く、壊れやすい
Robot Framework	ライブラリしだい	キーワード（表形式）＋ Python	Selenium・Playwright・Appium などを包む層	読みやすい。中身は包んだライブラリに依存
PyAutoGUI / SikuliX	Windows・Mac・Linux	Python, Java	座標・画像認識（掴み方C）	どこでも動くが壊れやすい
ノーコード／RPA	製品しだい	録画・ブロック	内部は上のいずれかの仕組み	書き手を選ばない。中身が独自形式だとAIや外部ツールと連携しにくい

AI

AIエージェントとの相性

ツール・方式	AIがスクリプトを書く	AIが直接操作する	ポイント
Playwright	◎ 情報が多く、生成精度が高い	◎ 公式のPlaywright MCP（部品の木をテキストで渡す）	Web なら現時点でいちばん AI と相性がよい
Selenium	◎ 情報が最も多い	○ コミュニティのMCPなど	古い書き方が混ざりやすい点に注意
Appium	○ コードは書ける。部品のIDは実機の情報が必要	○ MCPサーバーあり（公式・コミュニティ）	画面の木をAIに見せる仕組みとセットで使う
Espresso / XCUITest	○ コードは書ける	△ ビルドが必要で試行に時間がかかる	開発者がアプリと一緒に書く用途に向く
pywinauto / macOS AX	○	○ Windows向けMCPなど／AXの木をテキスト化すれば可能	このリポジトリの ax_dump.py が「木のテキスト化」の例
画像・座標	△	◎ computer use（画面を見てクリック）	どこでも動くが遅く、費用もかかる
非公開の独自形式	× 学習データにない（仕様書を渡せば一部可）	△ APIやMCPを公開すれば可	AIに使わせるには、仕様を文章で渡す・口を開ける工夫が要る

◎ とても向いている ○ 向いている △ 工夫が要る × 難しい （2026年時点の一般的な見立て）

Q&A

Playwright とは？

Pythonで書ける？ → はい、その理解で合っています

Microsoft製のブラウザ自動操作ライブラリ。TypeScript/JS、Python、Java、.NET の4言語に公式対応。

ただし、言語で少し差がある

テストランナー（@playwright/test、並列実行・UIモードなど）はNode.js版の機能。Python版は pytest と組み合わせる（pytest-playwright）。操作のAPIはほぼ同じ。

対象はWeb（ブラウザ）だけ

iOS・Android・Windows・Macのネイティブアプリは操作できない。そこはAppiumや各OSの仕組みの担当。

```
# 準備
pip install playwright pytest-playwright
playwright install      # ブラウザを取得

# 操作を録画してコードを作る
playwright codegen --target python URL

# test_login.py
def test_login(page):
    page.goto("https://example.com")
    page.get_by_label("メール").fill("a@b.c")
    page.get_by_role("button",
                     name="ログイン").click()
    expect(page.get_by_text("ようこそ")) \
        .to_be_visible()

# 実行
pytest
```

示唆（一般論）

壁打ちの論点：AI時代に選ばれやすい自動操作の条件

1. テキストのコードで書ける

AIが読める・直せる・差分をレビューできる。録画やブロックだけの形式は不利。

2. 公開されていて情報が多い

AIの生成精度は、世の中にある例の量でほぼ決まる。

3. 画面をテキストで取り出せる

部品の木（DOM・アクセシビリティ）をテキストで渡せると、AIが部品を正しく特定できる。

4. AIから直接呼べる口がある

MCPなどで「操作」と「画面の取得」をAIに開くと、探索的なテストも任せられる。

問い：AIが「書き手」を代わるなら、ノーコードの価値は下がる。一方で、OSごとの違いを吸収する層や、安定して動かす実行基盤（端末の管理・並列実行・結果の記録）の価値は残るのでは？ ――どこを自前で持ち、どこを公開の仕組みとAIに任せるか。